RIPASSO

programma concorrente → due code, una macchina del caffè
programma parallelo → due code, due macchine del caffè



perchè usare sistemi concorrenti / distribuiti:
- efficienza
- per distribuire
- per condividere risorse
- per usare reti di computer
- per comunicare

ho n uomini che devono estrarre n palline, quante combinazioni saranno possibili ?

$(n*m)!/(m!)^n$

Un programma concorrente è un insieme di programmi sequenziali (processi) che si eseguono in parallelo e che comunicano

parallelismo: programmi che si eseguono su processori diversi, ma anche programmi multi-task che si eseguono sullo stesso processore (illusione di parallelismo)

comunicazione: scambio di dati, sincronizzazione via memoria condivisa o via scambio di messaggi

Esecuzione di un programma concorrente:
- insieme di processi (programmi sequenziali)
- ciascun processo dispone di un insieme di istruzioni atomiche
- ogni processo ha un 'puntatore' verso la prossima istruzione da eseguire
- l'esecuzione del programma concorrente è un arbitrario intreccio delle istruzioni dei vari processi (interleaving)
- durante un'esecuzione, la prossima istruzione è dunque scelta tra quelle puntate dai vari processi → tanti scenari possibili

I processi hanno un accesso comune alla memoria, che avviene tramite variabili semplici condivise che possono essere lette o scritte
- bisogna fare attenzione a come viene gestito l'accesso a queste variabili: transazione atomica o non atomica
- vari casi possibili:
  - → variabili in lettura e scrittura per più processi "Multiple Reader/Multiple Writer"
  - → variabili in lettura per più processi ma in scrittura per un solo processo "Multiple Reader/Single Writer"

Un processo è un programma (statico) in corso di esecuzione (dinamico). La sua esecuzione necessita un ambiente: uno spazio di indirizzamento dedicato, input e output.

Più processi possono eseguirsi sulla stessa macchina in un modo quasi simultaneo SE il sistema rispetta il tempo condiviso o è multitask.

Il sistema operativo è responsabile dell'allocazione delle risorse (memoria, input/output, ...)
→ un processo non ha accesso allo spazio di indirizzamento degli altri
→ illusione del parallelismo

In C un processo viene creato grazie alla funzione fork: pid_t fork(void);
Questa funzione crea un processo figlio e restituisce un identificatore del processo (PID):

- fork restituisce 0 per il figlio
- fork restituisce l'identificatore del figlio al padre

- pid_t getpid(void); → restituisce il PID del processo corrente
- pid_t getppid(void); → restituisce il PID del processo padre

Le variabili NON sono condivise tra padre e figlio, e, quando viene creato il figlio, lo spazio di indirizzamento del padre viene copiato.

Per eseguire un programma (di solito al figlio) si usa la funzione execvp

```
int execvp(const char *file, char *const argv[]);
```

L'array argv contiene il nome della command nel primo campo, gli argomenti nei campi seguenti, e deve finire con NULL.
Non crea nessun nuovo processo: lo spazio di indirizzamento viene rimpiazzato con quello del nuovo programma.

COMUNICAZIONE FRA PROCESSI

Per comunicare tra processi, si possono usare dei "tubi" (pipe): sono canali unidirezionali considerati come dei descrittori di file. Un processo può scrivere a un'estremità e un altro può leggere dall'altra.
Poichè sono considerati come descrittori di file, quando un processo crea un figlio, quest'ultimo recupera la tabella con i descrittori del processo padre e ha accesso al tubo.



La funzione int pipe(int fds[2]) crea un tubo e mette le sue due estremità nell'array fds. Si scrive su fds[1] e si legge su fds[0].
Dopo il fork, entrambi i processi possono scrivere e leggere, ma affinchè funzioni correttamente, è necessario chiudere il "lato" non utilizzato.

CONDIVIDERE VARIABILI

Per farlo si usano dei processi leggeri → Thread
Un thread è un filo d'esecuzione all'interno di un programma. Il tread è eseguito da un processo; un processo può gestire più thread, e si parla quindi di un processo multi-thread.
Ogni thread presenta i seguenti attributi:

- un puntatore di esecuzione o Program Counter (PC)
- uno stack d'esecuzione

In C esiste una libreria 'classica' per la creazione e la manipolazione di thread: la libreria POSIX. Per usarla, bisogna includerla nel codice: `#include <pthread.h>`

Creazione di thread

- Per creare uno thread, si usa la funzione:

```
int pthread_create(
    pthread_t *thread,  //Per ricordare i dati del nuovo thread
    const pthread_attr_t *attr, // Attributi dello thread
    void *(*start_routine) (void *), // Funzione da eseguire
    void *arg); // Argomenti della funzione
```

- In pratica, per gli attributi, si può mettere `NULL` per utilizzare gli attributi predefiniti
- Questi attributi permettono, per esempio, di definire una politica di ordinamento per lo scheduler
- Il codice eseguito dallo thread è dato dalla funzione `start_routine`
- Appena la funzione `pthread_create` termina, lo thread è creato e può iniziare l'esecuzione

Creazione di thread in pratica

- Si scrive una o più funzioni contenenti il codice da eseguire dai thread. Il prototipo deve essere:

```
void *function(void *)
```

- Per passare degli argomenti alla funzione al momento della creazione dello thread, si inseriscono come ultimo argomento della funzione `pthread_create`
- A volte è necessario attendere la fine dell'esecuzione degli thread creati.
  - se il programma principale termina prima, allora gli thread vengono distrutti
  - per questo scopo si può usare la funzione

```
int pthread_join(pthread_t thread, void **value_ptr)
```

  - Il primo argomento corrisponde a quello creato da `pthread_create`
  - `value_ptr` è il valore ritornato dallo thread chiamando la funzione

```
void pthread_exit(void *value_ptr)
```

  - Attenzione: chiamare questa ultima funzione provoca la terminazione dello thread!!!

Si può usare una variabile globale o passare il loro indirizzo come argomento di pthread_create. Le variabili devono essere dichiarate come "volatile".

ISTRUZIONI ATOMICHE

un'istruzione atomica viene eseguita completamente senza essere interrotta
La correzzione dei programmi dipende dall'ipotesi di atomicità

esistono diversi casi:

- registri atomici → operazione di lettura e di scrittura atomiche
- test-and-set → si può testare e modificare una variabile senza essere interrotti
- swap → si può scambiare il valore di un registro locale e di uno condiviso in un modo atomico

INTERLEAVING (semantica di intreccio)

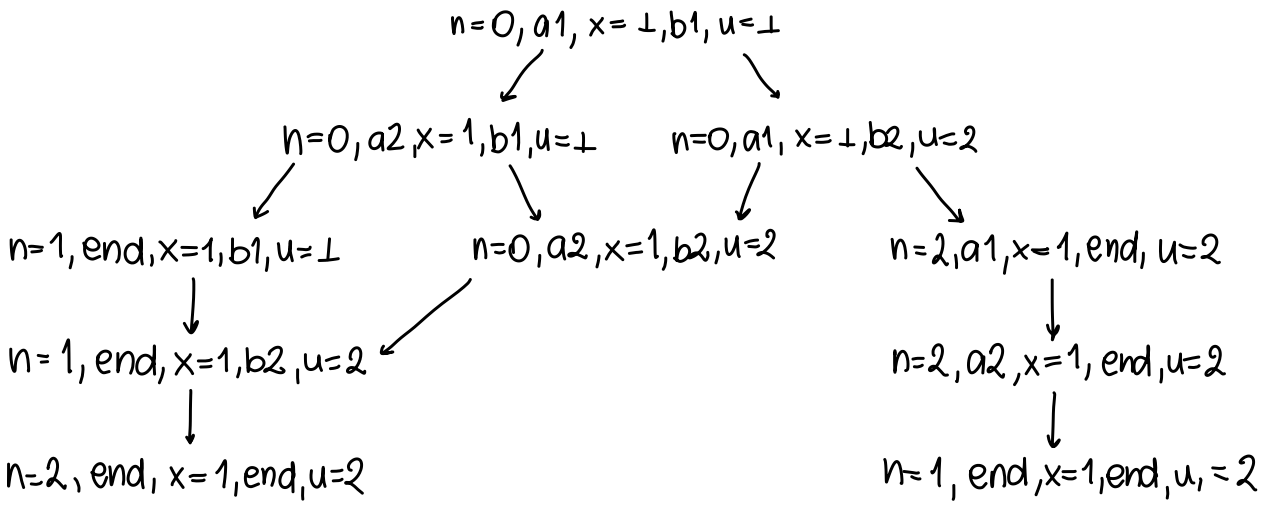
Uno stato di un programma concorrente P è una tupla che include:
- un puntatore alle istruzioni per ciascun processo
- un valore per ciascuna delle variabili (locali e condivise)

esempio: (n=0, end, x=1, b1, u,2)

Per due stati s1 e s2, si scrive s1→s2 quando è possibile passare da s1 e s2 utilizzando una delle istruzioni puntate in s1
esempio: (n=0, a1, x=1, b1, u=2) → (n=2, end, x=1, end, u=2)

Il diagramma degli stati di P è il sistema di transizione (ovvero il grafo) $TS_p = (S, \rightarrow, s0)$, dove:
- S è l'insieme degli stati di P
- → è la relazione di transizione inclusa in S×S
- s0 è lo stato iniziale di P

Un cammino in TS_p è uno scenario possibile per P



Una semantica ragionevole:
- per i sistemi con un processore condiviso fra più processi c'è una successione di istruzioni di un processo, poi di un altro, ecc.
- per i sistemi con più processori, ogni processo è legato ad un processore; la semantica d'intreccio non corrisponde esattamente alla realtà (ci sono delle azioni parallele), ma è corretta se non ci sono conflitti sulle risorse (evitare l'accesso simultaneo ai dati condivisi)
- per i sistemi distribuiti (diversi computer) non ci sono variabili condivise, ma dei canali di comunicazione; è molto diversa dai primi due casi, ma la semantica rimane interessante se si aggiungono l'invio e la ricezione di messaggi

MUTUA ESCLUSIONE

Permette di garantire che, in un determinato momento, un processo ha un accesso esclusivo a delle risorse comuni.
N ≥ 2 processi eseguono in modo continuo due sequenze di istruzioni:
- una sezione non critica (SNC)
- una sezione critica (SC)

In tal modo che la mutua esclusione sia garantita in SC, in ogni istante c'è al massimo un processo in sezione critica.

Proprietà da verificare:
- Mutua esclusione: al massimo un processo in SC ad ogni momento
- Assenza di deadlock: se più processi provano di accedere alla SC nello stesso momento, non devono essere bloccati nel pre-protocollo e almeno un processo deve riuscirci
- Assenza di starvation: se un processo desidera accedere alla sua SC, prima o poi ci riuscirà
- Attesa limitata: se un processo deve attendere per accedere ama sua SC, è possibile limitare il numero di volte in cui dovrà concedere il posto a un altro processo tramite una costante

Attesa limitata → Assenza di starvation → Assenza di deadlock

MUTEX e VARIABILE CONDIZIONALE

Per proteggere l'accesso ai dati condivisi, in C si possono usare dei lock (mutex).
Come funziona:
- un thread che desidera accedere a un dato condiviso richiede il lock
- Se il lock è libero, il thread procede
- Se il lock è preso, il thread rimane bloccato finchè il lock non viene liberato
- Quando ha finito di accedere ai dati condivisi, libera il lock

Questi lock sono condivisi fra i threads

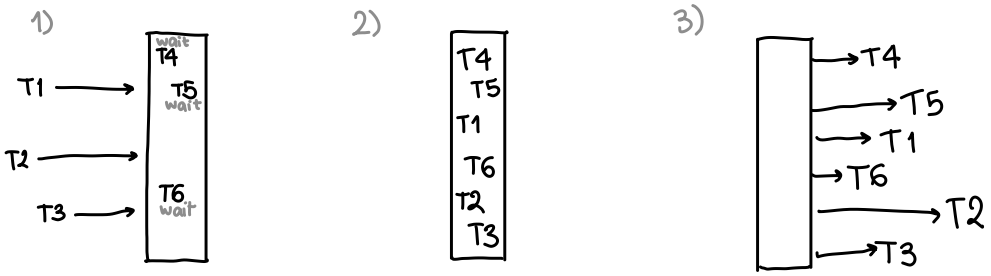
La libreria POSIX ha lock di tipo `pthread_mutex_t lock;`
Per iniziare un lock ci sono due modi:
- `lock = PTHREAD_MUTEX_INITIALIZER` (solo alla dichiarazione)
- `pthread_mutex_init(&lock, NULL)`

Prendere il lock → `int pthread_mutex_lock(pthread_mutex_t *mutex);`
Liberare il lock → `int pthread_mutex_unlock(pthread_mutex_t *mutex);`
} returnano 0 se esito positivo

Distruggere il lock → `pthread_mutex_destroy(&lock)`

BARRIERE

Sono un meccanismo di sincronizzazione utilizzato per esempio nel calcolo parallelo
Rappresenta un punto di sincronizzazione per N thread:
- i thread che arrivano alla barriera aspettano gli altri
- quando tutti gli N thread arrivano alla barriera, allora possono proseguire




```
pthread_barrier_t bar;
pthread_barrier_init(&bar, NULL, 5);
pthread_barrier_wait(&bar);
```

numero di threads che devono arrivare